School of Computing
FACULTY OF ENGINEERING AND
PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

Integrating Patch-Based Synthesis into a Building Application

Yik Hei Kwok

Submitted in accordance with the requirements for the degree of
BSc Computer Science

2024/2025

COMP3931 Individual Project

The candidate confirms that the following have been submitted*:*

<As an example>

Items	**Format**	**Recipient(s) and Date**
Final Report	*PDF file*	*Uploaded to Minerva (29/04/25)*
Scanned participant consent forms	*Image*	*In Appendix D of the Final Report*
Link to online code repository *(contains executable program)*	*URL*	*Sent to supervisor and assessor, (29/04/25)*
User Manual	*Readme.md*	*In online code repository*
Demo Video	*URL*	*Sent to supervisor and assessor, (29/04/25)*

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of student)

© 2025 The University of Leeds and Yik Hei Kwok

Summary

This paper describes the process of developing a building application on Unity, allowing users to define blocks with custom textures synthesised using a Patch Based Synthesis Method provided by Lin Liang et al. (2001). The steps for implementing the synthesis model and incorporating the model into the building application are outlined. Design decisions and management choices when developing the application are also outlined with justification. A comprehensive evaluation is conducted on the synthesis algorithm in regards to both time efficiency and texture quality through the use of KD trees and changing parameters.

Throughout the project, the patch-based synthesis algorithm and the building application has been completed successfully with minimal bugs.

The findings from this project show that using the KD method and GPU computation improves the time efficiency of the synthesis. However, the KD method specifically decreases the output texture quality.

On the other hand, using exceedingly large patch sizes will produce inconsistent colour transitions, while smaller patch sizes will lead to smoother transitions but can introduce noise. Meanwhile, a larger boundary region will improve texture accuracy, especially for regular patterns.

Acknowledgements

I would like to acknowledge the assistance that was given by my project supervisor, Rafael Kuffner dos Anjos, who has provided valuable insights throughout the duration of this project, and these insights were extremely helpful in the development of the texture synthesis algorithm.

I would like to also thank my assessor, Toni Lassila, for approving my project plan and also giving advice on potential evaluation techniques.

Lastly, I would like to thank my family and friends for emotional support during stressful times of the project.

Table of Contents

Table of Contents

Summary ..... **iii**
Acknowledgements ..... **iv**
Table of Contents ..... **v**
Chapter 1 Introduction and Background Research **1**
1.1 Introduction 1
1.2 Literature Review 1
1.2.1 Texture Generation Without a Source 1
1.2.2 Texture Synthesis with a Source 2
1.2.3 Types of Textures 3
1.2.4 Previous works on Texture Synthesis 3
1.2.5 The Patch Based Synthesis Algorithm 5
1.2.6 Optimization Algorithms using KD Trees 7
Chapter 2 Methodology **8**
2.1 Solution Design 8
2.1.1 Why Patch-based Synthesis? 8
2.1.2 Comparing Boundary Regions 8
2.1.3 The Distance Metric 9
2.1.4 KD Nearest Neighbour Algorithm 10
2.2 Project Decisions 11
2.2.1 Project Management Method 11
2.2.2 Version Control 11
2.2.3 Testing 12
Chapter 3 Implementation and Validation **13**
3.1 General Details 13
3.1.1 Implementation Description: 13
3.1.2 Software Architecture 13
3.1.3 Use of GPU Computation 15
3.2 Synthesis Implementation Details 15

3.2.1 Storing Previous Patch boundary regions 15
3.2.2 The KD Tree 16
3.2.3 The Distance Metric Computation 16
3.3 The Game Implementation 17
3.3.1 Game Mechanics 17
3.3.2 Implementation Details 17
3.3.3 The GUI 18
3.3.4 State Machines 19
Chapter 4 Results and Evaluation 21
4.1 Evaluation Measurement Criteria 21
4.1.1 What is tested 21
4.1.2 Testing Execution Time 21
4.1.3 Testing Texture Quality 21
4.2 Performance Results 22
4.2.1 KD Time Performance 22
4.2.2 CPU vs GPU Processing Performance 23
4.3 Texture Quality Results 25
4.3.1 Texture Quality between KD and Non-KD algorithms 25
4.3.2 Texture Quality with different parameters 27
Chapter 5 Conclusions 29
5.1 Conclusions 29
5.2 Future work 29
List of References 31
Appendix A Self-appraisal 33
A.1 Critical self-evaluation 33
A.2 Personal reflection and lessons learned 34
A.3 Legal, social, ethical and professional issues 34
A.3.1 Legal issues 34
A.3.2 Social issues 35
A.3.3 Ethical issues 35
A.3.4 Professional issues 35
Appendix B External Materials 36
Appendix C Images / Tables 37
Appendix D User Consent Form 40

Chapter 1
Introduction and Background Research

1.1 Introduction

Texture synthesis is the generation of new image textures of varying sizes based on pre-existing image textures. Texture synthesis is useful in many regards, especially in situations when multiple variations of a single texture is needed to produce a more natural texture environment.

Texture synthesis, despite its versatility, has on-going issues that many researchers are trying to solve. Texture synthesis algorithms rarely can produce naturally looking textures and be time-efficient at the same time. Therefore, a balance between these two aspects has to be achieved; otherwise, texture will either look unrealistic, or render too slowly for any practical use.

In this project, focus will be directed on the patch based texture synthesis algorithm provided by Lin Liang et al. (2001), and implementing the synthesis method into a house building Unity application. The intend of the application is to provide an immersive way of demonstrating how a working texture synthesis, which produces randomized textures, can be applied to a functioning program. Further justification will be given for choosing patch based synthesis over other synthesis methods, along with the evaluation of this patch synthesis algorithm concerning texture quality and time efficiency.

1.2 Literature Review

1.2.1 Texture Generation Without a Source

In the past, there are a multitude of algorithms that allow textures to be produced entirely by mathematical functions, without referencing a single source image.

From research provided by the University of North Carolina [1], it is stated that this type of texture generation is advantageous in terms of memory usage, as it automatically generates an image rather than storing images in processors for reference. However, the execution time of texture generation without a source is considered greater due to the necessity of computing textures using a step-by-step approach rather than directly applying or referencing a pre-existing image.

Through the same research, a simple example of this generation method is provided where a simple code snippet is used to create a black and white checkerboard pattern from scratch using U and V registers for defining texture coordinates, and D registers for defining the scale factor.

A more sophisticated example of this generation is provided by Julie Dorsey and Pat Hanrahan [2], whom attempted to generate metallic textures like copper. They achieved this through generating textures on different layers where the layers will have varying thickness through changing parameters.

However, research provided by Greg Turk [3] states that “creating a new texture without a source requires a programmer to write and test code until the result has the right look”. This indicates that no single algorithm can be used to create a wide variety of textures, making such generation methods inflexible.

1.2.2 Texture Synthesis with a Source

This type of texture synthesis shares similarities with the previous generation technique in the aspect that both dynamically produce textures creating texture variations. However, this type of texture synthesis uses pre-existing sample images to compute a new texture instead of purely mathematical functions.

Texture synthesis is often used in terrain generation. A terrain is described as a chunk of land usually of very large size. Storing a pre-existing large image as texture for the terrain will take up huge amounts of memory, and is infeasible if terrain isn’t pre-computed.

Research conducted shows an example application of texture synthesis to build up a terrain [5]. They firstly extracted patches from an existing sketch map of the terrain containing important terrain features using a Polygon breaking algorithm. Afterwards, important features from the patches like branch points are identified, and are mapped onto an existing terrain model through patch merging and coordinate mapping. This shows that a small sketch map is sufficient to build up the entire terrain.

However it’s stated by Alexei A. Efros and Thomas K. Leung [4] that texture synthesis may not generate textures with effective results if the source texture is too small due to the lack of features. Therefore a source texture should always strive for a balance between storage size and the amount of feature details.

1.2.3 Types of Textures

There are a vast majority of textures existing in the real world, and these textures are classified into different categories. Research conducted by Yanxi Liu et al. [6] classifies textures into regular, irregular, and stochastic classes. From figure 1.1, examples of these types of texture classes are shown.

Regular or near regular textures are textures which has an obvious pattern that can be visually determined with minimum amounts of randomness. Stochastic textures, being the complete opposite, shows no regular patterns and are extremely random in nature. Other textures that fall in between are irregular textures, as even though some patterns are shown, there are still obvious variations.

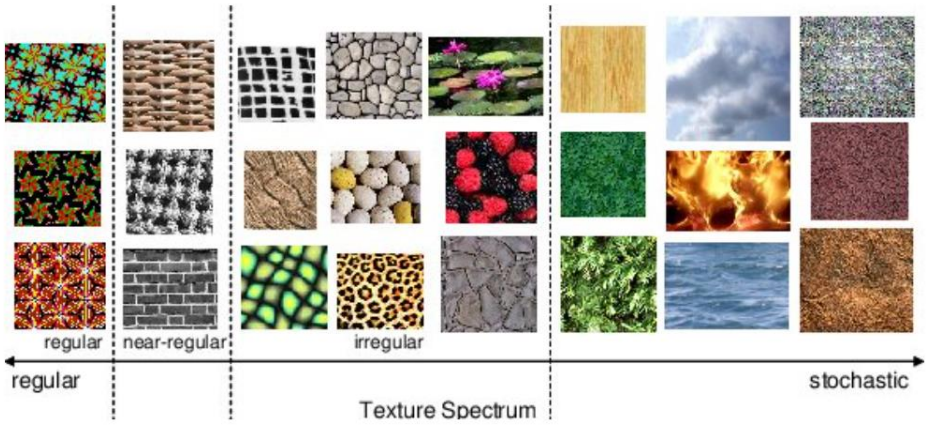


Figure 1.1: Texture examples for each texture type [6].

1.2.4 Previous works on Texture Synthesis

In this project, the focus of texture synthesis is targeted on the generation of new textures based on a source texture. Many texture synthesis algorithms have been developed over the past decades, many which fulfils the requirements of this project. A few of them had been considered for this project.

Texture Particles Synthesis

J-M Dischler et al. have developed an algorithm regarding the extraction of texture particles [7]. A texture particle is defined as a texture feature of an image; an example of this can be a brick on a brick wall texture. This algorithm excels at synthesizing stochastic textures.

This method is rather unique compared to other methods researched because it doesn't follow any traditional texture synthesis pipeline, but rather uses a cellular texturing technique, which is a method of extracting texture particles from the source image and placing them on a new surface.

RGB quantization is firstly used to identify types of texture particles and each particle type is segregated into layers, resulting in layers of texture particle image sets. The goal after this is to place the particles into the resultant image in a way that feels natural. Morphological dilation is therefore used to calculate the spatial arrangements between texture particles on the source texture, ensuring that particles placed on the resultant image are spread accordingly based on the calculated values.

Problems with this approach

This method of synthesis doesn't work well on generating regular and structured textures. This method essentially uses the technique of bombing texture particles onto a new image, and the placement of these particles won't be placed regularly because there is no algorithm that dictates them to be placed that way.

Feature-Based Synthesis: The Voronoi Diagram Technique

Muath and Philip Dutre has developed a different way of conducting texture synthesis with the use of Voronoi diagrams [8]. Contrary to the previous technique, this method claims to work well with both regular and stochastic textures.

User will firstly choose a feature from the source texture. The chosen feature is compared to the full source texture based on a "CIE L*a*b*" colour space, and based on the colour differences, feature points on the source image are identified.

Based on the feature points, a Voronoi diagram is created by dividing the source texture into polygons with each feature point being the centre of a polygon; the source image is segregated it into multiple smaller texture images based on the polygons. Each segmented texture therefore has a point placed on its centre [9].

A random point is first picked and its corresponding segmented texture is placed on the final texture. To choose the next feature points, each feature point is given a distance metric based on regularity standard deviation calculations. Feature points are selected as candidate points when their distance value is between the minimum and maximum distance value of each neighbour.

Problems with this approach

This synthesis method requires the user to manually select features from the source texture, meaning that human input is directly used in the process of creating the final image.

This method also only takes account of a single image feature. This method should work fine with simple images provided by the same research, shown in figure 1.2, it wouldn't work well with more complex images with multiple images, or images without obvious distinguishable features, as shown in figure 1.3.

Figure 1.2: The algorithm works well for textures where a single feature is obvious (the red flowers) [8]

Figure 1.3: The algorithm won't works well for textures without an obvious feature [20]

1.2.5 The Patch Based Synthesis Algorithm

Patch Based Texture Synthesis is a method where a source texture is segmented into smaller square patches, and the patches are used to construct a new image texture.

"Real-Time Texture Synthesis by Patch-Based Sampling" , published by Lin Liang et al. [10], has explained a simple yet efficient approach of implementing patch based synthesis. Their synthesis algorithm is mostly divided into two parts, extracting patches from a source texture and choosing patches for the final output image. This synthesis method is implemented in this project.

The way this algorithm extract patches from a sample image is similar to that of the previously mentioned Voronoi Diagram technique, but much simpler. Instead of requiring the user to choose feature points, the sample texture is simply sliced into square patches of equal size as shown in figure 1.4. The boundary zones of each extracted patch is also extracted and stored. Boundary zone of a patch refers to the area margins of predefined width residing on the edge of a patch, as shown in figure 1.5.

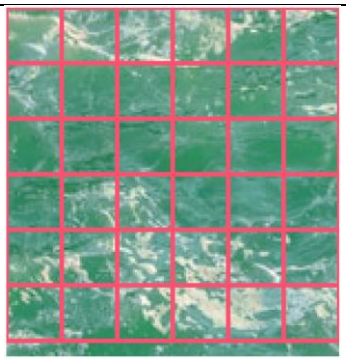


Figure 1.4: A texture sliced into multiple patches as shown with the red grid [20]

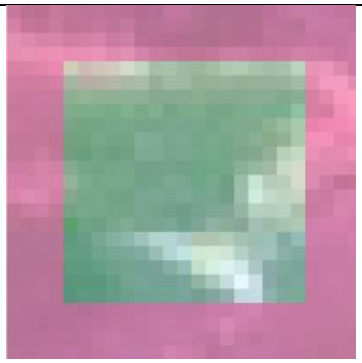


Figure 1.5: Boundary zone of a patch (pink shaded reigion) [20]

After the patches are extracted from the source image, the algorithm will start choosing patches. A random patch will firstly be chosen and placed on the top left of the final image. Afterwards, boundary zones of each extracted patch is compared to boundary zones of previously placed patches on the final image based on colour differences of each pixel; a distance value for each extracted patch is calculated after the comparison.

Once the distance value of each extracted patch is calculated, these values are compared with a maximum tolerance value. The patches with distance values lower than the maximum tolerance value will be labelled as potential candidates. A potential candidate patch will be chosen at random to be the placed on the final image. This process will continue until the final image is filled up with patches.

Patch based synthesis is widely praised for its ability to balance its efficiency and texture accuracy based on given parameters. Research conducted by Yiping Wang et al. [11] states that time for synthesising a texture is faster when larger patch sizes are selected, while the quality of transition between patches is smoother when smaller patch sizes are selected, proving that this method can be used to balance quality and time required based on different scenarios.

Patch based synthesis is used in a myriad of applications, and is not only limited to creating variations of a pre-existing texture as demonstrated above. One prominent example of this is the use of this algorithm on Image Inpainting as described by research conducted by Tao Zhou and Brian Johnson [12], whom removed objects from an image and used patch based synthesis to fill up the missing parts of the edited image.

1.2.6 Optimization Algorithms using KD Trees

In regards to the method of choosing patches with the patch based synthesis algorithm as mentioned previously, comparing each pixel in boundary zones for every extracted patch is known to be computationally expensive. The computation cost will be greatly reduced if some patches can be excluded from comparison before the patch comparison commences.

The previous research paper provided by Lin Liang mentioned the use of KD-trees to accelerate the choosing patches algorithm. Research provided by Jerome H.Friedman et al. has researched and implemented an “Optimized KD-tree” that runs in logarithmic time. [13]

A few key terms are mentioned in this research. A discriminator key refers to a feature representing a data record; each data record is not limited to only one feature. The partition value refers to a defined value that splits the chosen discriminator key, resulting in a tree. Within the tree, each node represents a patch.

The optimized KD tree approach aims to ensure the tree has minimum number of branches, and that the search only has to go down one side of the tree, eliminating the need to search half of the tree. Their method achieves this by choosing the discriminator key with the largest range of values within the dataset. The partition value is chosen to be the median value of the chosen discriminator key range.

I have created an illustration as an example on how an optimized KD tree is built, as shown in figure 1.6 and 1.7.

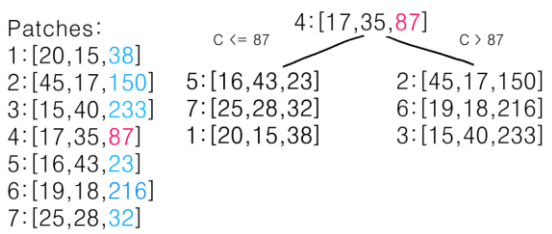


Figure 1.6: Assuming a patch is defined to have discriminators, width, height and colour, each defined with a numerical value, it’s clear that colour has the largest range of values. Therefore the discriminator key is set to colour and value 87 is the median of those values; the KD tree is split through the chosen partition value.

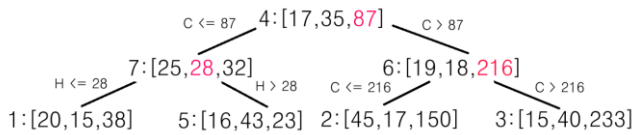


Figure 1.7: These steps are repeated in every iteration until an optimized KD tree is built.

Therefore, traversing either side of the tree can eliminate half of the total existing patches from the patch comparison process.

Chapter 2
Methodology

2.1 Solution Design

2.1.1 Why Patch-based Synthesis?

As previously stated, a patch-based synthesis algorithm is developed based on what Lin Liang's team (2001) has discovered [10], and I have chosen to implement this method of texture synthesis over other researched methods.

A good texture synthesis algorithm should be able to generate quality textures from both regular and stochastic source textures. The texture particles synthesis doesn't generate regular textures well because of its nature of bombing texture elements and dispersing them randomly.

While the Voronoi diagram approach should also be able to generate both regular and stochastic textures, as it follows a similar approach to patch based synthesis of combining patches, it only works well when the texture consists of only one feature. A texture synthesis technique that can be used on wider range of textures with more features is desired.

Overall, patch-based synthesis is chosen because not only is it a simpler approach to synthesising textures, it is also viable to produce a wide range of textures regardless of feature complexity.

2.1.2 Comparing Boundary Regions

In this project, the source texture is segmented into many patches based on the defined patch size. The top and left boundary regions are then obtained from each extracted patch, as shown in figure 2.1, and saved in a list before patches are being chosen.
The size of the boundary regions is based on a pre-defined value, which is a certain percentage of the patch size.

Figure 2.1: The unshaded parts are the left and top boundary regions. The yellow part is the area that will beplaced on image if this patch is chosen. [20]

After randomly choosing a patch to be the first patch of the final image, the right and bottom boundary regions of that

patch is saved as shown in figure 2.2, and the patch without its top and left boundary regions are placed on the first tile of the final image.

Since the right and bottom boundary regions of each chosen patch is stored, the top and left boundary regions of each patch can be compared with them to produce a distance metric for each patch. There are 3 cases, comparing only the left and right boundary, comparing only the top and bottom boundary, and comparing both sets of boundaries. The possible cases are shown in figure 2.3

Figure 2.2: Size of this patch including boundary regions is equivelent to size of the yellow area shown in figure 2.1. [20]

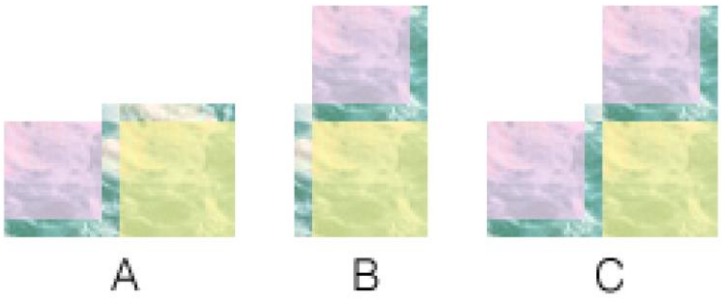


Figure 2.3: Patches shaded in pink are patches already placed on the final image. Patches shaded in yellow are patches being considered. [20]

A: When choosing patches for the first row of the final image, only left right boundaries is considered

B: When choosing patches for the first patch of each row after the first row, only top and bottom boundaries is considered

C: In any other case, both boundary sets have to be checked

2.1.3 The Distance Metric

A distance metric is calculated by comparing the colour difference of each pixel in the boundary regions of a patch, which is essential for choosing similar candidate patches to make up the final image. As mentioned, boundary regions of an already placed patch is saved and compared with boundary regions of each extracted patches, so that a distance metric is computed for each extracted patch.

From Liang's research, the distance metric equation is defined as

$$\left(\frac{1}{A}\sum_{k=0}^{A}\left(p_{B_k}^{i}-p_{out}^{i}\right)^2\right)^{\frac{1}{2}},$$

"A" refers to the number of pixels in the boundary region, and $p_{B_k}^{i}-p_{out}^{i}$ refers to the colour value difference of a pixel in the boundary region of an extracted patch and the previously placed patch.

After all patches have an associated distance metric, a maximum tolerance value is calculated using the equation $d_{max} = \epsilon \left(\frac{1}{A} \sum_{k=0}^{A} \left(p_{out}^{i} \right)^2 \right)^{\frac{1}{2}}$, where p_{out}^{i} refers to a pixel colour value of the boundary region of a previously placed patch, and ϵ is a randomness constant value. The ϵ value usually has a range from 0.1 to 0.4, depending on the category of texture; regular textures usually have a smaller ϵ value, while irregular or stochastic textures tend to have a larger one.

The proposed method, while not stated in the research, clearly uses the Ecludian distance approach to calculate distance metrics. I have decided to follow this approach in my project because of its proven effectiveness of deducing feature similarity between datasets and clustering compared to other distance metrics based on provided research by Archana Singh et al. [14].

The Manhattan distance, while is less costly, excels in Path Finding algorithms, which is not useful for texture synthesis. It also produces distortions when clustering data. The Minkowski distance is also proven to perform worse than the Ecludian distance in terms of time performance even though they eventually lead to similar results.

2.1.4 KD Nearest Neighbour Algorithm

As stated previously, the KD nearest neighbour algorithm excludes some patches from the patch comparison algorithm to reduce computational costs.

Liang’s team used a provided ANN library [15] to generate a KD tree for comparing boundary zone pixel values between previously placed patches and extracted patches. ANN supports the sliding midpoint rule, which attempts to set the partition value of the tree to the midpoint between the minimum and maximum value of the chosen discriminator key, so that the tree nodes are split evenly.

However, this project is developed in C# Unity, and ANN is a KD tree module implemented in C++, which therefore does not meet the project requirements.

There are a limited number of available KD tree modules that are applicable for C#, but the Super Cluster KD Tree module seems to fit the project’s requirements well. This module, written by ericreg, supports all basic requirements for this project’s use case, allowing the user to choose the number of discriminators to use and number of data points that should be returned.

2.2 Project Decisions

2.2.1 Project Management Method

This project is implemented using an Agile management approach, which means the project is managed using sprints.

Initially, a waterfall approach is considered, but due to the complexity of the texture synthesis algorithm, it is unfeasible to implement the entire algorithm first and attempt to test for all parts of the algorithm afterwards. With agile management, not only will the project be segmented into smaller and understandable steps, it will also tremendously reduce the required effort to debug problems in the synthesis.

The project further benefits from agile management when implementing the building application. Unlike the synthesis algorithm which follows a solid structure, new functions or ideas may be considered throughout the building application's development; agile development allows these ideas to be integrated flexibly throughout its development.

The project is managed using a Kanban Board, which contains the project backlog tasks of this project. Within the duration of the project, 4 sprints are planned over the 2 months of development, with each sprint being roughly 2 weeks. Sprint goals are defined for each sprint, and a selection of backlog tasks relevant to the sprint goals are selected as To-Do at the start of every sprint. The Kanban Board during the progression of sprint 3 is shown in figure C.1 at Appendix C

Testing is conducted on each completed task in the To-Do list to ensure that all tasks within the sprint have no errors or bugs which could affect the progress of the next sprint.

2.2.2 Version Control

Version control is needed for this project to make sure version back-ups are always available just in case the current version of the application encounter issues. Git is used as the primary tool for version control, as it allows multiple versions of the application to be saved and referenced for future use. A new version of the application is committed using Git when a small feature is implemented with no bugs. A commit message is given for every commit describing what has been accomplished.

2.2.3 Testing

The Texture Synthesis Algorithm

The texture synthesis algorithm uses manual testing because there isn't a correct answer to output textures. Bugs in the texture synthesis algorithm is usually not obvious at first glance since there can be imperfect transitions even when there are no issues with the algorithm.

A customized texture as shown in figure 2.4 is therefore created to ensure problems with the synthesis algorithm can be detected.

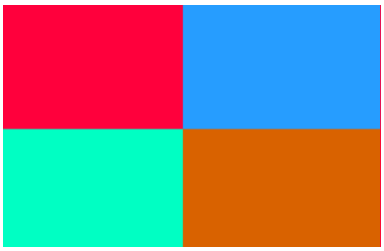


Figure 2.4: The texture used to debug the synthesis algorithm

This texture has purposely distinct colours. The synthesis algorithm is considered faulty when colours of the final output texture are all over the place even at a low ϵ randomness value, and correct when large patches of the same colour sticks together, shown in figure 2.6 and 2.5 respectively. In addition, patches and their boundary zones are also saved as images to ensure they are extracted correctly and didn't contribute to potential artefacts in the final image.

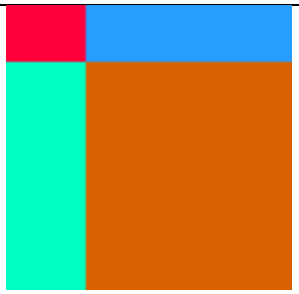


Figure 2.5: Texture generated when there are no bugs in the algorithm
(There can still be colour transitions because there may be patches with a mix of 2 colours.)

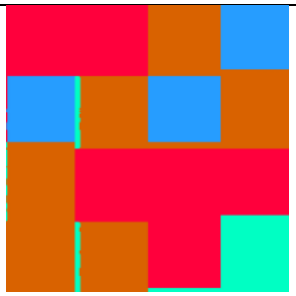


Figure 2.6: Texture generated using an algorithm with existing issues / bugs

The Building Application

The building application is a small scale application, and functions of the building application can easily be tested simply by switching through different scenes of the game. Testing different parts of the building game is therefore unrepetitive and can also be done through manual testing.

Chapter 3
Implementation and Validation

3.1 General Details

3.1.1 Implementation Description:

As explained previously, a block based building application is built, which allows users to create custom cube blocks with different synthesised textures.

The software is created using C# Unity, using mostly a mixture of the System.IO file class provided by Microsoft and Unity's own Compute Shader class. System.IO is used for basic logic operations that requires less processing power. The Compute Shader class is used for more graphic demanding operations that requires the GPU for efficient processing.

The implementation workflow of the project is segmented mainly into two parts; the implementation of texture synthesis, and the integration of synthesised textures into the building application.

3.1.2 Software Architecture

While C# is regarded as many to be an object oriented programming language, and forces code to be encapsulated within classes, the program is written in both procedural and object oriented programming manners depending on the situation.

Procedural Aspects

The "Texture Synthesis" class initiates the synthesis by calling a Start Synthesis function, which read pixels from a source image, segments the source image into patches, and segments the top and left boundary regions of the derived patches.

The "Choose Patches" class is responsible for choosing candidate patches by boundary region comparison with previously placed patches, producing a synthesised image. To maintain code readability, some functions called by this class are delegated to a "Process Patches" class.

Even though the synthesis algorithm is segmented into multiple classes, its functions are called sequentially instead and each class inherits Unity's default MonoBehaviour. Therefore, even with the use of classes, the synthesis algorithm is still procedural in nature.

Object Oriented Aspects

Each time a new cube is created, its corresponding texture atlas is loaded, processed as a texture and loaded into a new material. Since multi-sided cube textures are not supported in Unity by default, the cube also has to be UV mapped for the cube surface to follow the texture atlas arrangement [16].

Throughout the development of the application, it is noticed that cube creation occurs when previewing cube textures, processing terrain generation, updating inventory, and placing new blocks. Given that cube creation has a lot of complex steps, duplicating the related code in multiple places will have a detrimental impact on code readability and quality, and may even lead to scalability issues.

To mitigate this, a Cube Class is custom defined, which is designed to encapsulate processes for creating a new cube. The components controlling parts of the game that requires cube creation inherits from the Cube Class. Within these components, base.createCube() is called, which starts the cube creation process.

A class diagram is illustrated, showing the relationship between the cube class and components that inherits from it, as shown in figure 3.1.

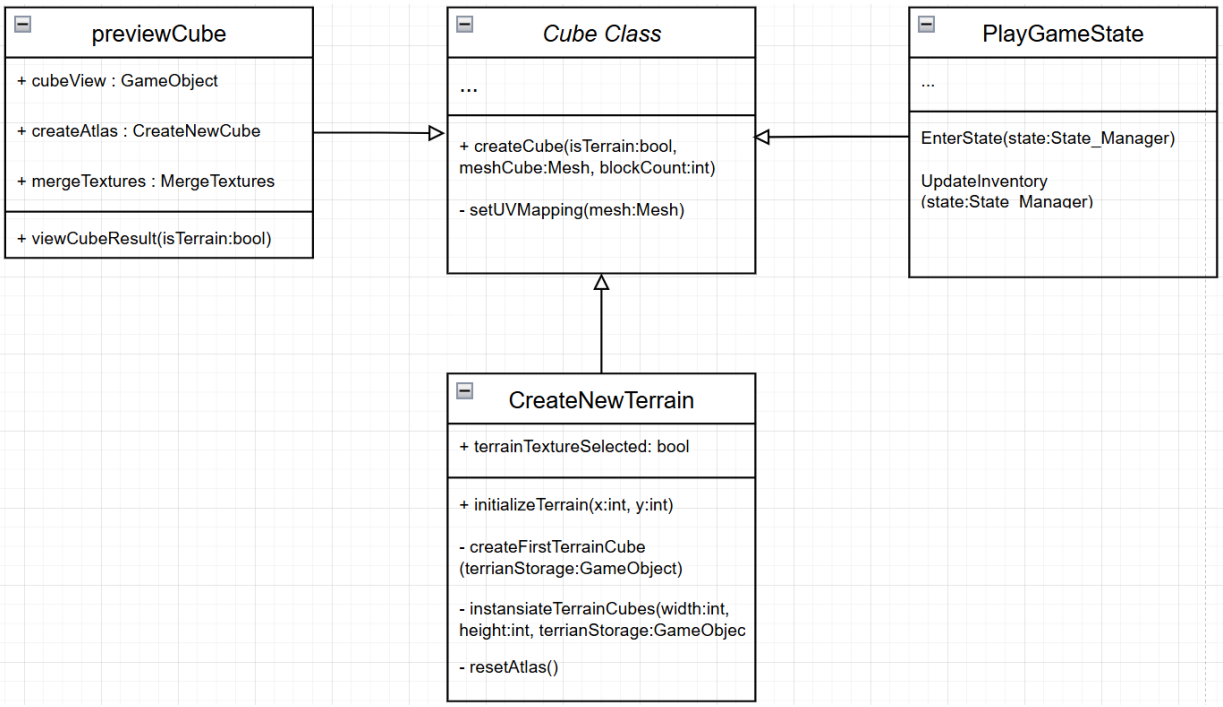


Figure 3.1: Class Diagram representing the inheritance relationship between the cube class and the inherited components.

3.1.3 Use of GPU Computation

The texture synthesis algorithm is computationally expensive, since individual pixels in a source image or boundary region of patches have to be analysed, stored, and compared in various parts of the algorithm, requiring huge amounts of parallel calculations.

The CPU is usually reliable for sequential tasks for general purposes, but has limited number of processing cores, resulting in limited parallelism.

In the case where individual pixel values have to be compared and calculated, GPU processing will likely result in greater computational speed due to its superiority parallel processing abilities. Offsetting pixel computation from the CPU to the GPU allows more pixels to be processed in a batch at the same time.

Unity has a built in Compute Shader which allow developers to offset repetitive calculations to the GPU, enabling greater parallel processing capabilities. The Compute Shader has been used extensively in this project, handling tasks such as reading pixel values from images, and comparing pixel values between boundary regions. The effectiveness of using GPU computation will be analysed in the results section.

3.2 Synthesis Implementation Details

3.2.1 Storing Previous Patch boundary regions

The final image is produced by placing patches starting from the top left corner, processing row by row, and ending at the bottom right corner. Every time a new patch is placed into the final image, the right and bottom boundary regions of that patch will be stored for later comparison when choosing the next batch of candidate patches.

The implementation for storing the right boundary region is straightforward. When a new patch is placed, the right boundary region pixels of that patch are saved in an array and used for the next comparison. This approach works because the right boundary region is only relevant when selecting the patch that will be placed immediately to its right.

The implementation for storing the bottom boundary region is more complex. Since patches are placed from left to right before progressing to the next row, the bottom boundary region of a placed patch becomes relevant only when comparing patches positioned in the next row, as shown in figure 3.2. Therefore, a 2 dimensional array is defined, with each element storing the bottom boundary pixels of a placed patch in the current row. When choosing patches for the next row, the array will be referenced for boundary comparison.

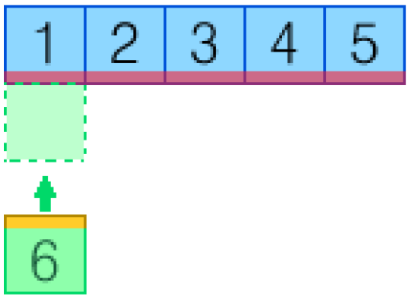


Figure 3.2: Number is the order of which patches are placed in the final image. Bottom boundary region of patch 1 is only relevant when patch 6 is placed (assuming there are only 5 patches per row).

3.2.2 The KD Tree

If the goal of using the KD tree is to exclude patches from boundary comparison, it is sensible to exclude patches that have greater dissimilarities compared to previously placed patches while keeping similar ones for comparison.

The KD tree is generated based on distinctive properties of each individual patch. However, due to the limited features that are different between each patch, a simpler approach was used, which is to use the average colour of a patch as the only discriminator for constructing the tree.

The average colour of a patch is calculated by adding all pixel values of that patch and dividing the sum by the number of pixels. The most similar third of the patches with most similar average colour are selected for comparison, and the other two-thirds are discarded.

Within the implementation of the game, the synthesis algorithm without KD trees are used. This is because using the KD algorithm potentially leads to a worsened effect on texture quality. The effects of this KD filtering will be discussed in the next section.

3.2.3 The Distance Metric Computation

Since patches are compared with previously placed patches in the same way regardless if KD methods are used, both KD and non-KD methods use the same functions defined for the distance metric computation.

I have specifically separated the patch comparison mechanism into 3 parts.

The first part involves the computation of the distance metric for each patch, which is done by comparing the top and left boundary regions of each patch with the bottom and right boundary regions of previously placed patches.

The second part involves the computation of the maximum tolerance using boundary regions of previously placed patches.

The last part involves comparing the distance metric value of each patch with the maximum tolerance, and label patches whose distance metric is lower than the maximum tolerance as candidate patches. One candidate patch is randomly selected as the chosen patch.

3.3 The Game Implementation

The building application uses the patch based synthesis model, but has a completely independent from the synthesis development process. In this game, the user will be given the ability to place created blocks onto other blocks to create structure. This game is highly inspired by Minecraft, which is a very popular block building game.

3.3.1 Game Mechanics

Blocks in the game are categorized as either terrain blocks or placeable blocks. Terrain blocks are used to create the default terrain where the player can stand on and build. Placeable blocks are blocks that will show up in the user's inventory, which can be placed on any type of block when selected.

Once the terrain is generated, the user can move around the terrain and place blocks if placeable blocks have been defined by the user. The user can define new placeable blocks by invoking the GUI system responsible for creating new textures. An example of a built structure in the game is shown in figures C.2 and C.3 at Appendix C.

The game shares many similarities with Minecraft, so the control schema of this game is based on Minecraft's control schema for the most part. [17]

3.3.2 Implementation Details

Controls

Controls of the game is developed using Unity's new input system, which uses an event based algorithm. Inputs are defined in an Action Map, which stores all input actions used in the game. Each input action is bounded to one or more keys, and these actions will be called when corresponding keys are pressed to provide different functionalities.

Terrain Generation

Users are asked to input a width and height before generating the terrain, and these parameters represent the number of blocks that make up the terrain. The terrain is

generated by firstly creating a new block using the cube creation method as described in section 3.1.2, and then instantiating new blocks based on the first block in a grid like fashion where each new block is placed right beside a previously placed block.

Ray-casting

A ray is emitted starting at the centre of the player's camera and extending outwards, and its used to detect blocks set on the Tile layer. When the ray reaches a block, the side of that block which the ray hit can be determined by calculating the dot product between the hit surface normal and the direction vectors of each block surface.

When holding a placeable block, a semi-transparent preview block is created next to the side of an existing block where the ray hits. The preview block will be replaced with the block selected in the inventory when right click is pressed.

Gravity and Collisions

The default Unity Rigidbody component is used to handle gravity of the player, which automatically creates gravity. Each block is given a collider to prevent the player from phasing through blocks. When the player switches to flying mode by double tapping the space key, the isKinematic parameter of the Rigidbody is set to false, effectively disabling gravity and allowing them to fly.

3.3.3 The GUI

The game primarily has 3 graphic interfaces displayed at different stages of the game: the Start Menu, Texture Selection, and Block Preview. These interfaces are shown in figure 3.3 to 3.5.

The Start Menu interface allow players to set the number of blocks that make up the terrain.

The Texture Selection interface allow players to use the patch synthesis algorithm on each side of a new cube. The player can choose the patch size, boundary size, and ϵ randomness value before generating a texture.

The Block Preview interface shows how the cube will look like after using texture selection on all sides of a cube. The cube shown can be rotated by holding the right mouse to view all sides.

The design of the GUI is meant to look simple with a black and white design to increase contrast and readability

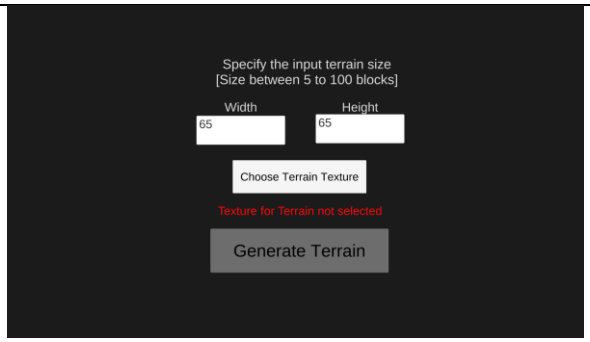


Figure 3.3: The Start Menu GUI

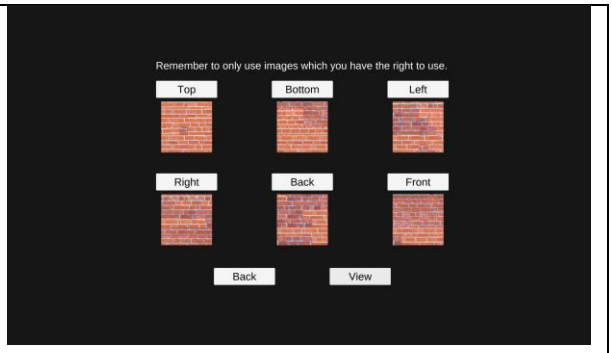


Figure 3.4: Texture Selection GUI

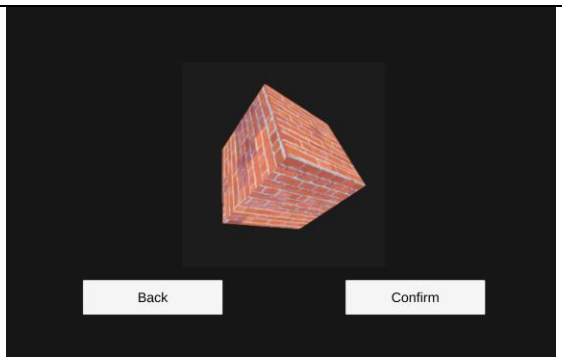


Figure 3.5: Preview Block GUI

3.3.4 State Machines

State Machines in Unity are used when there are multiple states in a game where each state encapsulates some differing behaviours within a part of the game.

Structure of the State Machine

A State Class containing “EnterState” and “UpdateState” abstract functions was set up, and all states defined inherits from this class; each state will have its EnterState and UpdateState override functions, meant to override functionality of the functions defined in the State Class.

A State Manager Class contain references to all states, and one state will be labelled as the current state; the current state can be changed using a “SwitchState” function. When the current state is changed, the “EnterState” function of the new state is invoked, and its “UpdateState” function will be called every frame until a different state becomes the current state.

Two State Machines are used in the game, one for setting up terrain generation, and another for the actual building game. These two state machines have some states that work very similarly.

State Machine 1: Terrain Generation Setup

There are 3 states for the state machine managing terrain generation, Start Menu State, Choose Texture State, and Choose Preview State.

In the Start Menu State, the terrain generation menu shows up, and listeners are added to the terrain size input fields and all other buttons in the menu. Clicking on the “Choose Textures Button” switches the current state to the Choose Texture State.

In the Choose Texture State, the menu for selecting Textures shows up, and textures for each side of the cube can be chosen. Clicking on the “Preview Cube button” switches the current state to the Choose Preview State.

In the Choose Preview State, the player can see a preview of the cube. The Confirm button will bring the user to the Start Menu State with the terrain texture set. Only when the terrain texture is set can the player generate the terrain by clicking on “Generate Terrain”.

State Machine 2: Building Block Application

There are also 3 states for the state machine managing the building game, Play Game State, Choose Texture State, and Choose Preview State.

Every time the user enters the Play Game State, the atlas folder is traversed, and new blocks will be added to the user inventory if new atlas images exist. If the user presses on the M key, the state will be switched to the Choose Texture State.

It should be noted that the Choose Texture State and Choose Preview State of this state machine functions similarly to the previous state machine, and the only difference being the file location of where the texture atlases are saved.

A State Machine diagram is created to summarize all the states described above, as shown in figure 3.6.

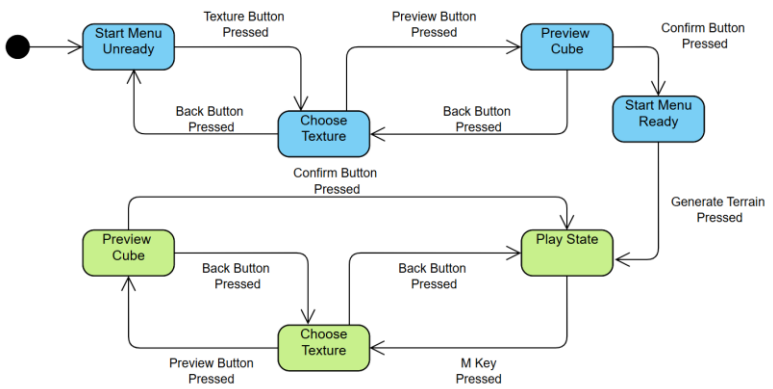


Figure 3.6: State Machine diagram showing all states in the game. Blue states represent states for Terrain Generation Setup. Green states represent states for building block application.

Chapter 4
Results and Evaluation

4.1 Evaluation Measurement Criteria

4.1.1 What is tested

An effective texture synthesis algorithm should output textures of high quality, yet is able to synthesize it within a feasible amount of time. Therefore, the execution time and the quality of the implemented texture synthesis algorithm will both be tested and evaluated upon.

4.1.2 Testing Execution Time

The execution time of the algorithm can be affected by a multitude of factors including the patch size, boundary size of patches, size of the final image, the use of GPU processing, and the use of KD trees. The execution time analysis for this project will be focused on the use of KD trees and the GPU computation.

The execution time is measured using the System.Diagnostics.Stopwatch C# class, which allows the measurement of execution time in different parts of the program. The results will be displayed in various diagrams created using Matplotlib.

4.1.3 Testing Texture Quality

From Liang's research [10], its stated that using an optimized KD tree on the algorithm could reduce the output texture quality. Therefore, analysis is conducted to evaluate if the use of the KD tree truly has such effect.

Furthermore, the variations of the patch size, and boundary region size can also affect texture quality. Changes between these parameters will therefore be tested to further understand how they will affect texture quality.

There is considerable difficulty in measuring quality of textures using a programatic approach because sythesised textures can highly differ from the source image even if transition between patches are coherent. Therefore, cognitive test are established instead, where a selection of university students is asked to complete a test created using the Tkinter python library, and to respond to a series of questions to evaluate the quality of textures.

4.2 Performance Results

In order to truly understand the performance efficiency of the algorithm, test environment specifications is recorded before performance benchmarks are conducted.

Test Environment Specifications

CPU		GPU	
Specs	Values	Specs	Values
Number of Cores	14	Number of Cores	5888
Number of Threads	20	Boost Clock (MHz)	1035-1485 MHz
Base Speed	2.50 GHz	Memory Size	8 GB GDDR6
Cache L1 L2 L3	1.2mb 11.5mb 24mb		

4.2.1 KD Time Performance

The time efficiency of the KD method is important, as there is no reason to use it if its time efficiency is even worse than the original algorithm without the KD tree.

I have created timers within the code to record the time efficiency of the algorithm with or without the KD method. The test only records the time required for choosing patches, as it's the only part of the synthesis where these two methods differ in.

Both algorithms are run 10 times with 10 different output sizes to show how an increasing output size affects the time needed for choosing patches. The patch size, overlap size, the ϵ randomness and the source texture are kept the same throughout the experiment. The execution time results are recorded and plotted out to show the trend.

Results (shown in figure 4.1)

The time used for choosing patches is lower when using the KD method compared to without using the KD method. This trend is expected because using the tree eliminates two-thirds of patches from boundary region comparison.

It also seems that the time required for choosing patches increases as the output size becomes larger. This trend is also expected because a larger output image is made out of more patches, therefore the patch comparison algorithm has to be executed more frequently.

The time performance difference between the algorithm with and without the KD method increases when the output size increases. This is likely due to larger output images requiring more patches, and the KD method is called each time a patch is to be chosen. Therefore the cumulative time saved from the KD method is much greater for larger output images.

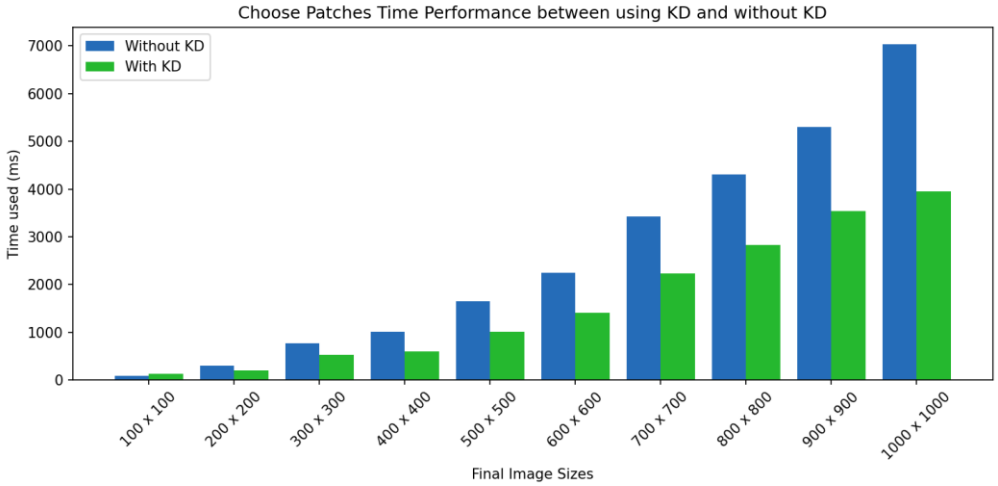


Figure 4.1: Plotted graph regarding total time needed to choose all patches for the final image with both algorithms, at different output image sizes.

4.2.2 CPU vs GPU Processing Performance

Unity's Compute Shader can offset parallel calculations for GPU processing, which can potentially be faster than using just a CPU. The validity of this statement is tested.

The implemented synthesis algorithm relies heavily on GPU processing using Compute Shaders. However, a CPU computation approach is developed for a small part of the algorithm alongside its GPU counterpart to understand if using the GPU truly improves time efficiency. I have chosen the patch extraction process as the part of the algorithm for comparison.

Both methods are run 4 times with 4 different input image sizes. The patch size, overlap size, ϵ randomness value , and the output image size are kept constant throughout the whole experiment.

Results (shown in figure 4.2)

The CPU implementation is straight forward. A loop is initiated which loops through every single pixel in the source image one at a time, and allocates the pixels into patches based on pixel position in the source image. This computation method should be relatively slow due to the lack of parallel processing.

The old GPU method involves pre-computing the required number of patches, and calling the compute shader on each patch. Every time the compute shader is called, the source image is passed into the shader. The pixel values of a patch is stored in the compute buffer and added to a pre-defined array. The end pixel position of a processed patch relative to the source image is also saved, which serves as the starting point for the next patch to be extracted.

It is suprising to see the original GPU computation method performing worse than the CPU computation method by a great margin. From further analysis, its seems the method perform worse due to the cost of calling the compute shader. Invoking the compute shader is costly because pixel data is passed into and out of the compute shader every time it's called, which may lead to unnecessary overhead costs.

Due to the unexpected results of the original GPU method, an attempt to create an new GPU method is made which calls the compute shader only once. The source image is passed into the shader, and it is segmented into patches, where pixels from all patches are stored sequentially in one array.

The new GPU method is proven to reduce the computational overhead and processing time by a huge margin compared to the old GPU method. This method truly takes advantage in its parallel processing capabilities, resulting in a greater overall time efficiency compared to the CPU method.

The patch extraction process time increases with an increase in source image size because there are more pixels to be processed for larger images.

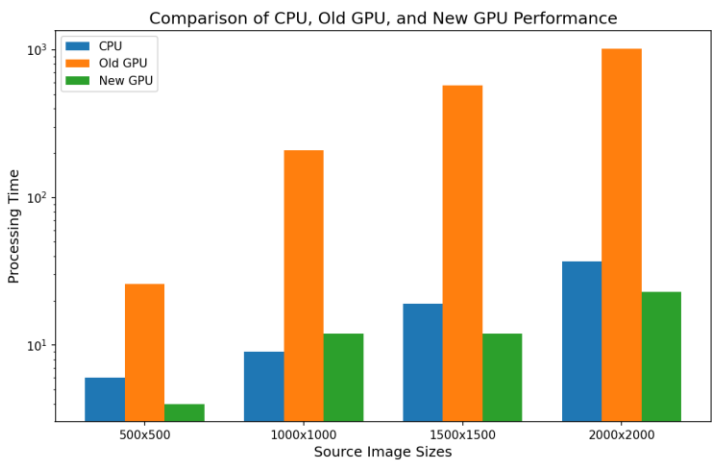


Figure 4.2: Graph regarding total patch extraction time using different CPU/GPU methods at different input image sizes

4.3 Texture Quality Results

The quality of textures are determined based on cognitive perception from people due to the unquantifiable number of parameters that determines texture quality.

4.3.1 Texture Quality between KD and Non-KD algorithms

It has been explained that using a KD nearest neighbouring search over patch similarity can potentially cause a degradation in texture quality. This claim is to be validated.

Experiment Set-Up

A Tkinter Python application is developed consisting of 9 questions. For each question, there is a source texture image on the right, and 6 buttons displaying textures synthesised from the source on the left; 3 of the synthesised textures are produced using the KD tree, while the remaining 3 are not. The textures shown in Table C.1 of Appendix C are used for the questions, including Regular [R1,R2,R3], Irregular[IR1,IR2,IR3], and Stochastic [S1,S2,S3] categories. A screenshot of the Tkinter application is shown in Figure C.4 of Appendix C

Users are asked to order the synthesised textures from the most natural to the least natural by clicking on the texture that looks the most natural first, and the least natural last. To minimize the training effect, the order of texture category shown in each question are intentionally interleaved. Furthermore, a white screen is displayed for 2 seconds each time a question is answered, giving the user a brief mental reset before proceeding to the next question.

When users rank the textures for each question, the selection order of the 3 synthesised textures generated without the use of KD are recorded. A higher selection order grants higher points, and higher points mean that synthesised textures created using a non-KD method looks more natural than ones created using KD.

Results

A heat map is produced to show the scores obtained by each participant on each texture, as shown in figure 4.3. Tiles that are more blue indicates the non-KD method produces more natural textures.

Within each texture category, some textures are perceived to have worse results using the

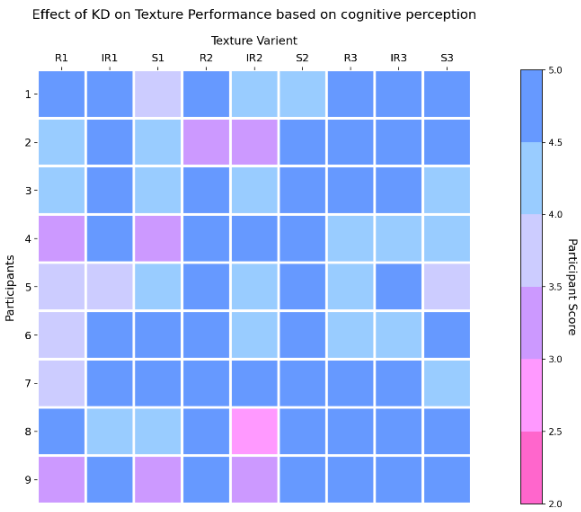


Figure 4.3: Heatmap showing the perceived naturalness score of textures generated by non-KD method for each participant on different textures

KD method, while others suggests no such pattern. Therefore the category of texture has little effect on the participant's cognitive perception on whether textures are generated using the KD method.

Overall, textures produced using the KD method is generally perceived to be less natural compared to those produced by the original method. This observation is expected, because using the KD method evaluates fewer patches during the patch selection process, potentially filtering out better matches. However, this also suggests that relying solely on an average colour discriminator for the KD tree does not lead to an effective KD algorithm in terms of texture quality.

More participants seem to find textures produced from R1, S1, and IR2 to be of similar quality regardless if KD trees are used. I have revisited some participants to understand why they found it difficult to distinguish between the two methods, and they claimed that textures generated from these sources lack natural transitions regardless of which method is used.

It was later realized that wrong parameters were given when generating textures from R1 after the experiment concluded, leading to terrible texture quality regardless of the method used. Therefore data from R1 is excluded from the evaluation.

S1 is a sky texture with many clouds, and IR2 is a texture filled with bananas. They both may not yield good results from either synthesis method due to complex gradients and cloud patterns in S1, and similar colours and messy placement of bananas in IR2. Clouds are meant to be have messy yet coherent shapes, and bananas are meant to have curve and regular shapes. However, neither method of choosing patches have knowledge about shapes and only align patches using colour. Therefore, the generated shapes of objects within these textures are incoherent regardless if the KD method is used, resulting in equally terrible textures.

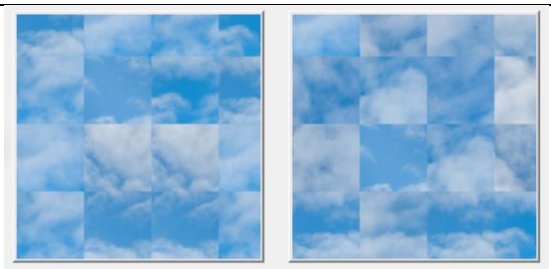


Figure 4.4: Generated Textures for S1, left is generated with KD, right is generated without KD.

Figure 4.5: Generated Textures for IR2, left is generated with KD, right is generated without KD.

4.3.2 Texture Quality with different parameters

As stated previously, research provided by Yiping Wang et al. [11] has outlined some observations regarding how changing parameters in the algorithm affects texture quality, and they will be proven or disproven based on results provided by the synthesis algorithm of this project.

Similarly, cognitive perception is used for evaluating textures. 5 individual university students were asked to partake in judging the textures results.

1. Larger patch sizes cause more inconsistent colour transitions between patches

A rainbow gradient texture created in Clip Studio Paint has been chosen for synthesis. Patch sizes 100, 250 and 400 are used on the same texture, with its boundary regions set to the same percentage of the patch size as stated in section 2.1.2. Each patch size configuration is run 3 times. The variations of the generated textures are shown in figure 4.6.

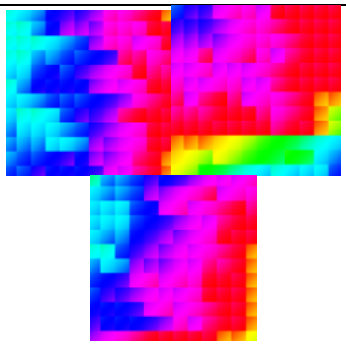
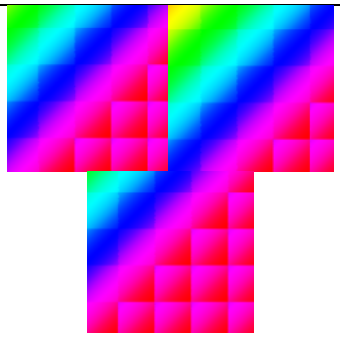
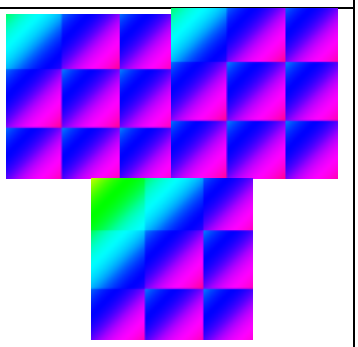
Patch Size 100	Patch Size 250	Patch Size 400
		

Figure 4.6: Gradient textures synthesised using different patch sizes (100, 250, 400)

4 of the 5 students preferred the texture with a 250 patch size, while 1 preferred the 100 patch size.

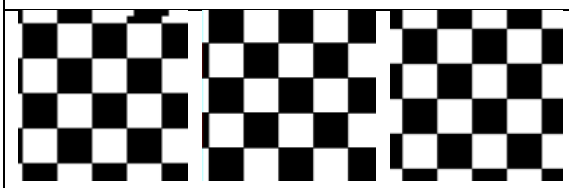
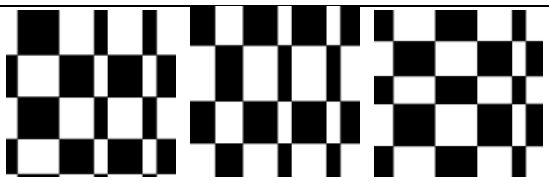
Setting a larger patch size on a source image with constant size results in fewer patches of larger size being extracted, which reduces the number of patches to choose from. Therefore, an inconsistent transition of colours is resulted when using larger patches due to fewer patches closely matching the previously place ones.

A student commented that “texture generated with 100 patch size transition colours well, but looks messy”. A small patch size will results in more patches of smaller size being extracted, therefore increasing the number of patches to choose from. Due to the ϵ randomness value, many patches with slightly deviating colours may be chosen, and these colour deviations causes distortions which results in a messy texture.

Therefore, a balance between minimizing noise and consistent colour transitions has to be made when choosing the patch size.

2. Unsuitable patches may be chosen when area of boundary regions are too small

The same checkers pattern suggested in Yiping Wang et al paper was used to determine effect of texture quality when using a larger and a smaller boundary size of 80 and 20 respectively.

Boundary region 80	Boundary region 20
	

All students preferred the results given by boundary region 80 due to its greater similarity to the original texture. The texture generated by boundary region 20 has compressed and disproportional checkers, and therefore shows less similarities to the source image.

With a smaller boundary region for comparison between patches, the patch comparison algorithm has less information regarding the appearance of each patch, which may lead to inaccurate transitions, especially regarding repetitive patterns.

However, using a larger boundary region for patch comparison does require more pixel to pixel comparisons that could increase the algorithm’s time efficiency. Therefore, it is advised to use a larger boundary region for regular textures, and a smaller boundary region for stochastic textures.

Chapter 5
Conclusions

5.1 Conclusions

Overall, objectives set out for this project were completed successfully. Algorithms relating to texture synthesis with a source image were researched thoroughly, and the patch based synthesis provided by Lin Liang has been implemented successfully.

The synthesis algorithm was incorporated into a building application using Unity successfully, where it is used for creating new textures for building blocks. The application performs efficiently with minimum latency issues, and the synthesis algorithm can generate 1000x1000 textures within seconds, which is scalable for the project's requirements.

Analysis of the synthesis algorithm has been conducted on both time performance and texture quality by changing various parameters, and results derived provides interesting yet expectable results. Results regarding texture quality could be improved upon due to a limited number of respondents.

From the evaluation of the synthesis algorithm, it is evident that the time efficiency of the algorithm improves when GPU parallel computation is utilized, and when KD trees are used to filter out patches. On the other hand, texture quality improves with a larger boundary region, without using KD trees, or when selecting relatively smaller patch sizes.

5.2 Future work

While the synthesis algorithm runs successfully, there are some edge cases the algorithm fails to deal with. The algorithm would run really slowly if its given an extremely small source image or when asked to compute an extremely large final image. This is likely due to optimization issues regarding choosing patches, which runs at $O(n^2)$ complexity. Further optimizations to this algorithm can be further explored upon, perhaps by incorporating other methods used in Lin Liang's research [10] like the Quad Tree Pyramid or Principal Component Analysis.

Currently, people can still differentiate textures created with the KD tree and ones that didn't, meaning that patches kept by the KD algorithm may not be best matches. This is likely because only an average colour discriminator is used to construct the KD tree. A potential

method to improve texture quality from the KD algorithm is to introduce machine learning algorithms for determining texture patterns, which can be used as an additional discriminator.

For the building application, synthesised textures are only used on cube block surfaces, and cannot be applied to other types of objects due to how the final image is synthesized. Research provided by Connelly Barnes et al. shows a brief introduction to how textures can be synthesized using a patch-based method, using a lazy approach, on 3D objects with unique shapes [18]. Using this approach in the building application would bring more variety to the building designs.

List of References

[1] Rhoades, J., Turk, G., Bell, A., Andrei State, Neumann, U. and Varshney, A. 1992. Real-time procedural textures *In*: *Proceedings of the 1992 symposium on Interactive 3D graphics - SI3D '92*. New York, New York, USA: ACM Press. Available from: https://dl.acm.org/doi/pdf/10.1145/147156.147171

[2] Dorsey, J. and Hanrahan, P. 1996. Modeling and rendering of metallic patinas. *International Conference on Computer Graphics and Interactive Techniques*. Available from: https://graphics.cs.yale.edu/sites/default/files/metallic_patinas.pdf

[3] Turk, G. 2001. Texture synthesis on surfaces In: Proceedings of the 28th annual conference on Computer graphics and interactive techniques. New York, NY, USA: ACM. Available from: https://dl.acm.org/doi/pdf/10.1145/383259.383297

[4] Efros, A.A. and Leung, T.K. 1999. Texture synthesis by non-parametric sampling. *Proceedings of the Seventh IEEE International Conference on Computer Vision*. **2**, pp.1033-1038vol.2. Available from: https://graphics.cs.cmu.edu/people/efros/research/NPS/efros-iccv99.pdf

[5] Zhou, H., Sun, J., Turk, G. and Rehg, J.M. 2007. Terrain synthesis from digital elevation models. IEEE transactions on visualization and computer graphics. 13. Available from: https://faculty.cc.gatech.edu/~turk/my_papers/terrain_synth_tvcg.pdf

[6] Liu, Y. n.d. A comparison study of four texture synthesis algorithms on regular and near-regular textures. Cmu.edu. [Online]. [Accessed 13 April 2025]. Available from: https://www.ri.cmu.edu/pub_files/pub4/lin_wen_chieh_2004_1/lin_wen_chieh_2004_1.pdf.

[7] Dischler, J.-M., Maritaud, K., Lévy, B. and Ghazanfarpour, D. 2002. Texture particles. *Computer graphics forum: journal of the European Association for Computer Graphics*. **21**(3), pp.401–410. Available from: https://onlinelibrary.wiley.com/doi/full/10.1111/1467-8659.t01-1-00600

[8] Sabha, M. and Dutré, P. 2009. Feature-based texture synthesis and editing using Voronoi diagrams In: 2009 Sixth International Symposium on Voronoi Diagrams. IEEE, pp.165–170. Available from: https://ieeexplore.ieee.org/document/5362369

[9] Anon n.d. What is a Voronoi diagram? Bristol.ac.uk. [Online]. [Accessed 13 April 2025]. Available from: https://www.bristol.ac.uk/maths/fry-building/public-art-strategy/what-is-a-voronoi-diagram/.

[10] Liang, L., Liu, C., Xu, Y.-Q., Guo, B. and Shum, H. 2001. Real-time texture synthesis by patch-based sampling. ACM Trans. Graph. 20, pp.127–150. Available from: https://people.csail.mit.edu/celiu/pdfs/TOG.pdf

[11] Wang, Y., Wang, W. and Wu, E. 2006. Optimizing the parameters for patch-based texture synthesis In: Proceedings of the 2006 ACM international conference on Virtual reality continuum and its applications. New York, NY, USA: ACM. Available from: https://dl.acm.org/doi/abs/10.1145/1128923.1128936

[12] Zhou, T., Johnson, B. and Li, R. 2016. Patch-based texture synthesis for image inpainting. arXiv [cs.CV]. [Online]. Available from: https://arxiv.org/pdf/1605.01576

[13] Friedman, J.H., Bentley, J.L. and Finkel, R.A. 1977. An algorithm for finding best matches in logarithmic expected time. ACM transactions on mathematical software. Association for Computing Machinery. 3(3), pp.209–226. Available from: https://dl.acm.org/doi/pdf/10.1145/355744.355745

[14] Singh, A., Yadav, A. and Rana, A. 2013. K-means with Three different Distance Metrics. International Journal of Computer Applications. 67, pp.13–17. Available from: https://research.ijcaonline.org/volume67/number10/pxc3886785.pdf

[15] Mount, D. 1998. ANN Programming Manual. https://www.cs.umd.edu/~mount/ANN/Files/1.1.2/ANNmanual_1.1.pdf

[16] Anon 2016. Change texture of cube sides. *Unity Discussions*. [Online]. [Accessed 13 April 2025]. Available from: https://discussions.unity.com/t/change-texture-of-cube-sides/82546/4.

[17] Anon n.d. Minecraft.net. [Online]. [Accessed 13 April 2025]. Available from: https://www.minecraft.net/en-us/article/minecraft-controls.

[18] Barnes, C. and Zhang, F.-L. 2017. A survey of the state-of-the-art in patch-based synthesis. Computational visual media. 3(1), pp.3–20. Available from: https://link.springer.com/article/10.1007/s41095-016-0064-2

[19] Anon n.d. Manytextures.com - Free seamless textures to download - JPG, PSD, PNG. Manytextures.com. [Online]. [Accessed 25 April 2025]. Available from: https://www.manytextures.com/.

[20] Brady Erickson 2019. Free textures, tutorials, and more. Texturelabs.org. [Online]. [Accessed 25 April 2025]. Available from: https://texturelabs.org/.

Appendix A
Self-appraisal

A.1 Critical self-evaluation

Before this project, I had no knowledge regarding any type of texture synthesis, and therefore was extremely conflicted if implementing a patch based synthesis algorithm was even practically viable. Thankfully, before choosing the synthesis method, I have done a large number of proper research in regards to different types of synthesis algorithms and Unity functions that can aid the development process. Therefore, I was able to choose a synthesis algorithm with moderate complexity and implement it successfully.

The most challenging part of implementing the synthesis algorithm has to be the array calculations when using Unity compute shaders. While using compute shaders are extremely efficient in pixel to pixel calculations, it does not accept two-dimensional arrays. Therefore, calculating placement of pixels in arrays is one of the most tedious tasks in the project. However, I persevered and was able to use it consistently throughout the algorithm.

The building application was surprisingly easy to implement. However, I did not understand the full benefits of using a synthesis algorithm before building the application, so the implementation of the building application doesn’t use the synthesis’s full potential. I should have understood the main objectives of using a synthesis algorithm to begin with before planning on how to use it in the game.

I have also faced issues with adhering to the duration of planned sprints, because some parts of the synthesis (notably at sprint 2) takes way more time than expected. This has significantly reduced the available time duration for the next 2 sprints. If better time management is used, I could have developed more features for the building application that can utilize the benefits of the synthesis more, such as a block rotation feature before block placement.

A.2 Personal reflection and lessons learned

I enjoyed working on this challenging yet manageable task. This project is my first time applying methodologies learnt over the years from multiple modules in this course, notably from Computer Graphics, Operating Systems, and Professional Computing, to a self-directed project.

From this project, I've learnt the importance of preparing a more detailed time schedule. Although I have managed the sprints of this project well with the use of a Kanban board, I have struggled to stay on track with the tasks during sprints. This is likely due to the lack of a proper timeline, which left me with too much time flexibility when working on each task. I feel like the project would benefit from a detailed timeline, like a Gnatt Chart.

I have also learnt a lot about incorporating research into a project. I have developed some personal projects in the past, but none of them use research provided by professional sources. I was never a reader and faced issues trying to understand different papers regarding synthesis algorithms, yet I persevered and was able to correctly implement methods provided by these papers and to prove or disprove claims made by certain papers.

Throughout the development of the synthesis algorithm, I have learnt a lot about parallel computation, which was used thoroughly in the synthesis algorithm. I have initially tried to loop through pixels of an image in a single for loop, but realized how slow it is when processing large images. I therefore used threading and parallel computation methods provided by Unity's Compute Shader, which were methods I've slowly understood over the project's duration.

Throughout the development of the building application, I have learnt a lot about Unity's 3D game making capabilities. The concept of meshes and materials in Unity were confusing concepts that I was eventually able to understand when developing the building application. I am sure understanding these capabilities will be of great benefit to me when I pursue game development careers in the industry.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

Two plugins are used within this project, the Supercluster KD Tree plugin and the Unity Simple File Browser plugin. Both plugins has MIT Licenses, which means users are allowed

to use them free of charge as long as the original licence and copyright notice are not deleted. These documents have been stored in the project.

Another legal issue may be caused when passing in images into the synthesis algorithm without the consent of the original creators. Therefore, I have chosen to use royalty free images provided by websites like Texturelabs and ManyTextures. It is stated that textures from these websites can be used for projects as long as it’s not used for monetary purposes.

A.3.2 Social issues

The building application is made as a sandbox game where players are given the freedom to build anything they want using custom textures. The given freedom to players, adding to the fact that unlimited number of structures can be built, may be an addicting experience to some people. This addiction may not be physically or mentally healthy for individuals who spend a lot of time in the application.

A.3.3 Ethical issues

While royalty free images are used in the development and evaluation of the building application, there is no certainty that individuals using the application will adhere to these moral principles. With so many images on the internet, users may pass in any image that looks nice into the application, which is a disrespect to intellectual property. To mitigate this issue, a warning message is displayed in the texture selection page, advising users to only use images they have the right to use.

Users do not need to submit any private information when using the building application, so no data privacy issues exist.

A.3.4 Professional issues

The project aligns with the rules given by the BCS code of conduct. There is no mis-leading information on what the final application can do or cannot do, and parts of the program that are not developed by myself are at least mentioned and credited in the report. The code quality of the project is well commented, especially on complicated parts of the synthesis algorithm. The building application does not harm or discriminate anyone when it is being used.

Appendix B
External Materials

Supercluster KD Tree

- Contributors: Ericreg, Corstian
- GitHub Repository: https://github.com/ericreg/Supercluster.KDTree

Unity Simple File Browser

- Contributors: Yasirkula, Joey Teng, Bclausdorff, Mingkyme
- GitHub Repository: https://github.com/yasirkula/UnitySimpleFileBrowser

UV Mapping Suggestion on Unity Cubes

- Solution provided by: Robertbu
- Unity Forum: https://discussions.unity.com/t/change-texture-of-cube-sides/82546

Application for Creating State Machine Diagram

- Application: https://online.visual-paradigm.com/diagrams/features/state-machine-diagram-software/;VPSESSIONID=C68BFB972B65D15829340D4487C0962A

Images used for Synthesis Algorithm Testing and Evaluation

Note: All example textures in this project comes from these sources

- Texture Labs: https://texturelabs.org/
- Many Textures: https://www.manytextures.com/

Kanban Application used for planning sprints

- Kanban flow: https://kanbanflow.com/

Class Diagram Creation

- Draw.io: https://draw.io/

Appendix C

Images / Tables

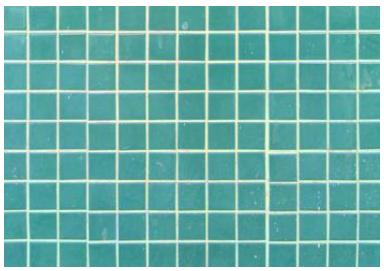
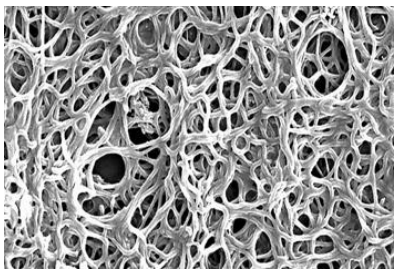
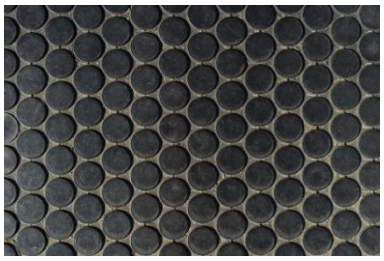
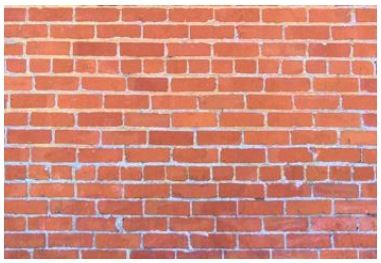
Regular Textures	Irregular textures	Stochastic Textures
R1 	IR1 	S1
R2 	IR2	S2
R3 	IR3	S3

Table C.1: Textures used for evaluating Texture Quality [19, 20]

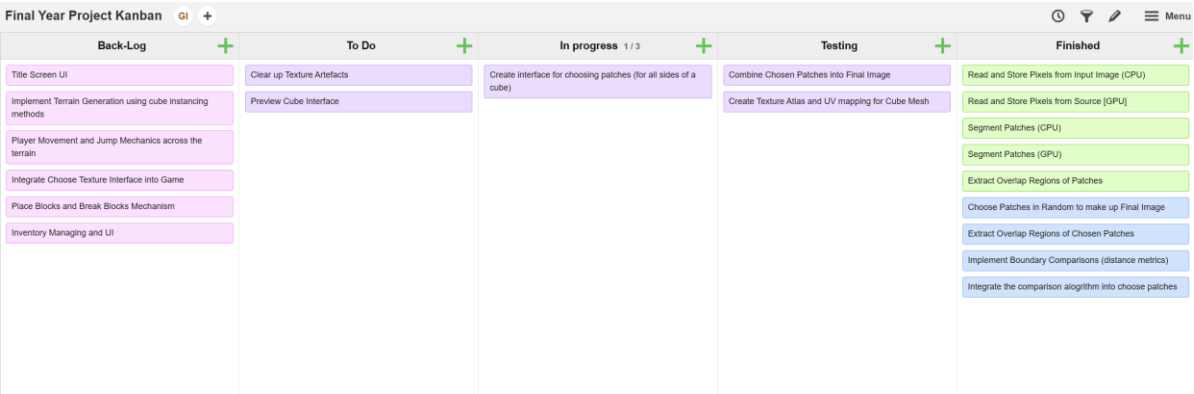


Figure C.1: The Kanban board progression during Sprint 3

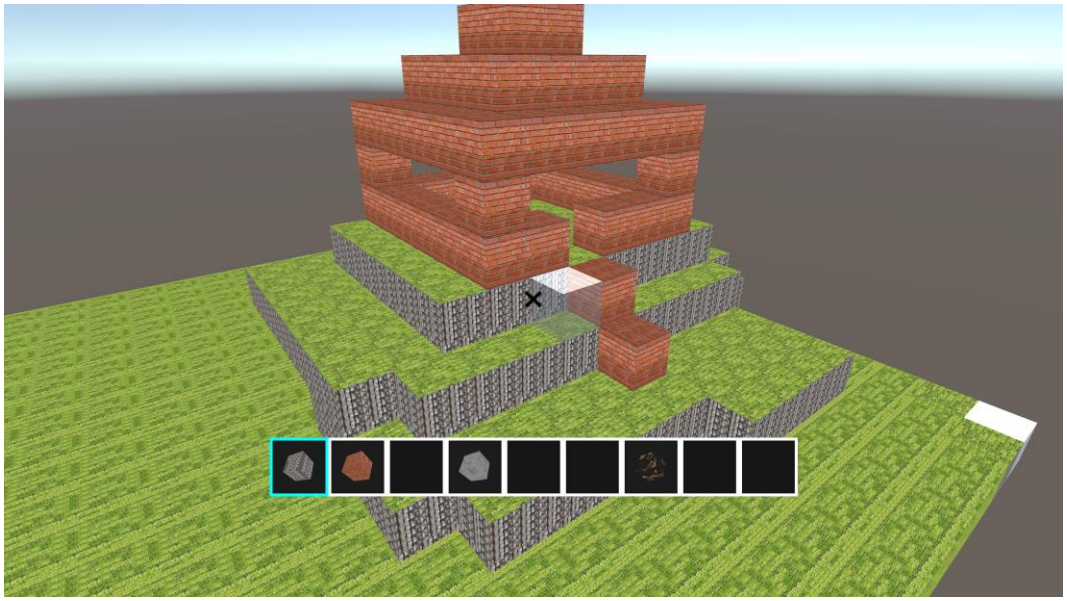


Figure C.2: A structure built in game

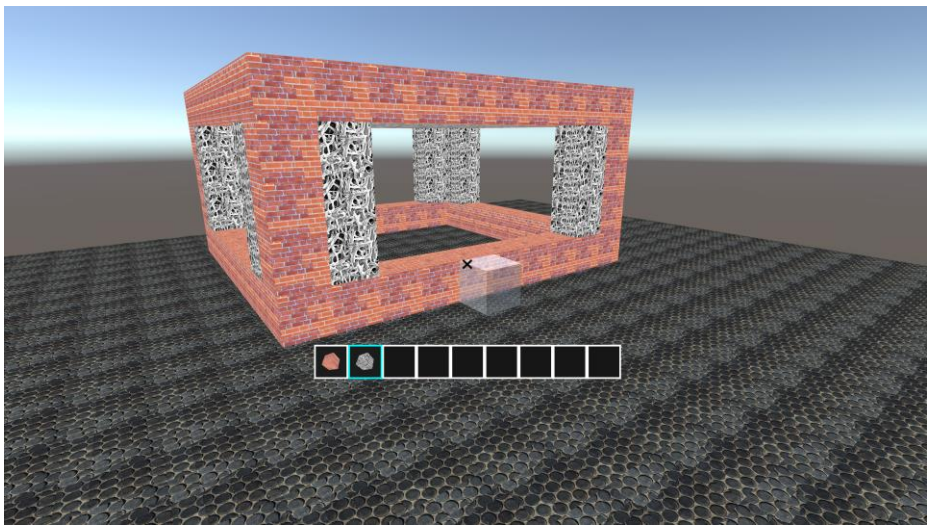


Figure C.3: Another structure built in game

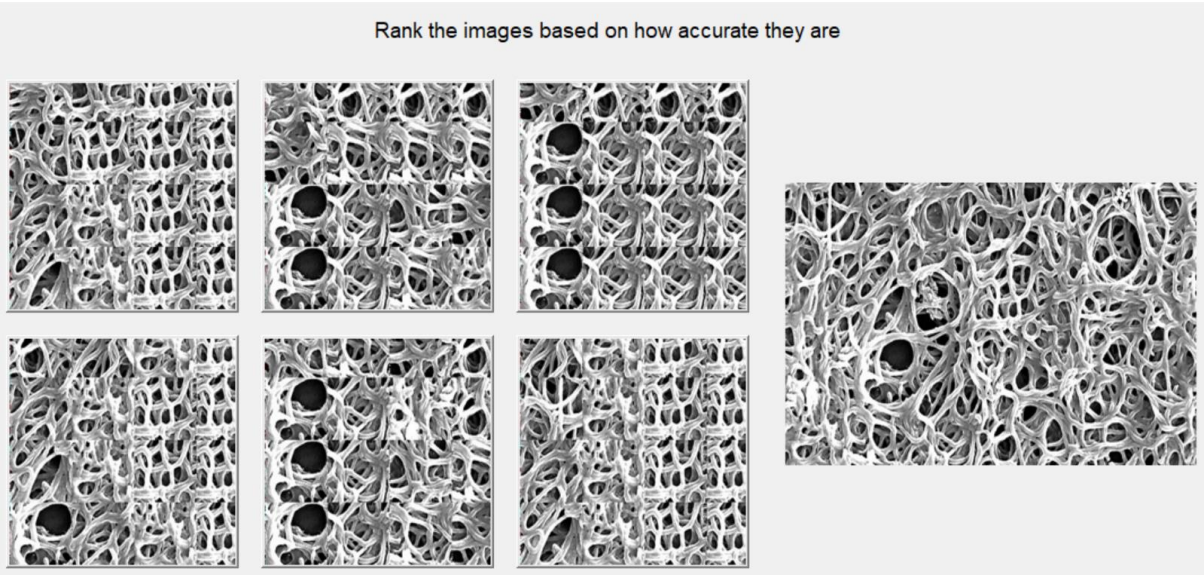


Figure C.4: Interface of a question for the KD Texture Analysis Tkinter Program

Appendix D
User Consent Form

User Consent Form

Evaluating Synthesised Textures

Good day to you sir/madam, you are being invited to take part in the evaluation of a texture synthesis project. Before participating, it is important for you to understand the purpose of this project, and what is to be evaluated. Please take your time to read this form, which contains all the information about this project and evaluation. Remember, you can always choose to not take part in this evaluation for whatever reason, as your participation is not mandatory.

Evaluation Aim:
This project is about the generation of synthesised textures through a patch-based synthesis algorithm.
The aim in the first part of the evaluation is to understand if the use of a KD algorithm in the synthesis can lead to a decrease in texture quality.
The aim in the second part of the evaluation is to understand how changing some parameters of the synthesised algorithm can lead to changing texture qualities.

Evaluation Process:
The full process of your participation should not last longer than 10 minutes.
For the first part of the evaluation, you will go through an application interface containing 9 questions. For each question, there will be a texture representing the original texture on the right, and 6 buttons containing synthesised textures on the left. You are expected to click on the texture that looks to most natural first, and the least natural last.

For the second part of the evaluation, 2 sets of unique textures with different synthesised results will be given to you. You will be asked which synthesized textures are preferable. Answers to this are more open-ended, and there won't be specifically planned question given to you.

Do you have to take part of it?
No. You are free to refuse participation or discontinue at any point during the process. After the first part of the evaluation, you will be asked if you would like to continue participating in the second part. If you choose not to, you are free to leave.

Personal Information:
No personal information will be recorded throughout the entire process, so there is absolutely no privacy concerns. The only recorded information regarding you is your opinions about the textures, and any opinion is kept anonymous.

Results of the Project:
Results collected from you will be published in a Computer Science Final Year Project Report that will be accessed by the School of Computing of the University of Leeds.